Demo Mode scenarios, explained in plain English

*Documentation only: nothing in the project was changed to write this. Every quoted log line below is a real line from a real run of that scenario in the simulator (`python3 sim/demo_server.py --dashboard`, Demo Mode, 22-23 October 2026). The complete log of each run is saved in `docs/scenario-captures/<scenario-id>.txt`, so you can check any quote. Everything here is **SIMULATOR** data: nothing on this page was measured on a real cluster, and all patient data is synthetic.*

Part 1. Read this first

1.1 Where the "activity log" is, and what you are looking at

- In the dashboard it is the **"Live activity"** box at the bottom of the **Overview** tab (it is visible in both the Simple and the Detailed view).
- **Newest line on top.** The screen shows the latest event first, so the story reads *upwards*. The numbered steps below read oldest-first (top to bottom), the way you would tell it.
- **Click any line** to open its raw details (the "pre" box with the numbers). You never need that on stage.
- The small times are the clock time of the line; in this document I use "+4.0 s" meaning "4 seconds after the first line of the scenario".
- Do not confuse it with the **Activity Log** *tab*: that tab (Detailed view only) holds the tamper check of the decision logs, the list of messages that failed verification and the Export button.

1.2 Why so many lines appear at once

1. **Four guards, four logs.** Four security agents (A, B, C, D) each write **their own** line for everything they see and do. One event in the real world becomes **four** nearly identical lines.
   In the app-compromise run each guard wrote 72-73 lines: A 72, B 73, C 73, D 73.
2. **They look all the time.** Each guard re-measures **all six** healthcare services continuously (every half second in the simulator, every second on a real cluster).
   A service that looks wrong gets a line from every guard every half second: in the first run, "Agent A sees records-api (C): 40 outbound connections..." appears at +0.0, +0.5, +1.0, +1.5 and +2.0 s.
   A service that looks normal gets a "sees ... as normal" line only about every 7-8 seconds.
3. **Everything is written down on purpose.** Every message that is sent, received, checked, voted on and acted on is logged. That is the point of the project (nothing happens secretly), but it is too much for a talk.
4. **The numbers:** across the 13 scenarios there were **6 to 23 lines per second** (typically about 10), and **55-70 %** of all lines were background chatter
   (Observation, Evidence Shared, Evidence Received, Evidence Score, "certificate held" and "executor rank" lines) that you can hide without losing the story.

Scenario (id)	Lines	Seconds	Lines per second
app-compromise	291	22	13.2
false-accusation	359	30	12.0
forge-evidence	175	8	23.3
agent-crash	209	21	9.9
transient-blip	96	11	9.1
slow-burn	217	13	16.7
ambiguous	170	14	12.0
validation-retry	338	37	9.3
failed-validation	298	50	6.0
lab-results-tampered	223	23	9.7
medication-failed-logins	235	23	10.4
database-cascade	285	23	12.4
obstruct	287	23	12.5

1.3 How to make the log readable in 10 seconds (the "Show" and "Agents" buttons)

Above the log there are two rows of buttons. **Every button starts switched on; clicking a button switches it off (and on again).** "Clear filters" switches everything back on.

- **Agents row** (A B C D): switch off three of them (leave only **A**) and the log is **four times shorter**, because the other guards repeat what A says.

- **Show row**: the 13 line types. For the presentation use one of these two recipes:
  * **Story only** (best for most scenarios): leave **on** *Agent Decision*, *Agreement Reached (3 of 4)*, *Action Taken*, *Warning*; switch **off** the other nine.
  * **Liar / forger scenarios** (false-accusation, forge-evidence, obstruct, ambiguous): leave **on** *Warning*, *Trust Change*, *Failed Verification*, *Investigate*, *Decision Withheld*, *Agent Decision*; switch **off** the rest.
- The search box accepts any word, e.g. `SUSPECT`, `isolated`, `FALSE POSITIVE`, `HUMAN REVIEW`.
- Switching to the **Detailed** view (top right) adds the Investigate tab, the "Who agreed" tables and the Activity Log tab. The **Simple** view is better on stage.
- **Known quirk of the Security Alerts tab (important, found while checking this document).** The dashboard builds that list from what it has seen since it was started, and **Reset does not clear it**. After "Reset" the first alert of the next scenario is named "C ... #1" again, so the old and the new run can be **mixed in one alert page**. Examples seen in my runs: after scenario 1 an old "C Electronic Health Records (EHR) API #1 ... RESOLVED" alert was still listed while scenarios 2, 3, 5 and 7 (which isolate nothing) were running; and in scenario 13 the alert's story box said "4 of the 4 security agents agreed (A B C D)" although the table beside it said "agreed by B C D" and the log line says "Quorum reached: B, C, D signed ...".
  How to stay safe on stage: (1) restart the dashboard before the presentation and, if you can, before the key scenarios (Ctrl-C, run `python3 sim/demo_server.py --dashboard` again); (2) for "who agreed" always trust the log line "Quorum reached: ... signed ..." and the table column "agreed by", not the number in the story box; (3) scenarios that isolate nothing (2, 3, 5, 7) create **no** Security Alert: that is the correct result, do not look for one.
- The **Agents** row of buttons reads **A B C D CTRL**; "CTRL" is the central controller of the centralized-comparison mode and is empty in these scenarios.
- Press **Reset** before every new scenario (the page says "press Reset before the next scenario").

1.4 Who is who: the "four security guards with walkie-talkies" picture

Imagine a hospital with **six apps** (the "services") and **four security guards**.

In the project	In the picture
The four **security agents**, named **A, B, C, D**	Four guards: guard A, guard B, guard C, guard D. Each has binoculars on **all six** apps and a walkie-talkie to the other three. They are identical; nobody is the boss.
The six **healthcare services** (workloads): **A** Patient Portal, **B** Login & Access Service, **C** Electronic Health Records (EHR) API, **D** Clinical Database, **E** Laboratory Results Service, **F** Medication Service	The six apps the guards protect.
Cutting a service off the network ("Isolation", a *quarantine network policy*)	Locking the door of that one app so it cannot talk to anything.
"Replace it from the **known-good image**"	Swapping the app for a fresh, factory-clean copy.
3 of 4 must agree ("quorum")	A door is locked only if at least three of the four guards raise their hands. One or two guards can never do it, and a guard who stays silent is never counted as "yes".

The letter trap. The same letters A-D name *guards* and A-D (and E, F) name *apps*. In the log:

- **"Agent A"**, "agent B", "A, B, C, D signed ..." = the **guards**.
- A letter at the end of a sentence, in brackets or after "contain/move/validate" (for example "contain **C**", "records-api **(C)**", "database **(D)**") = the **app**.
- So "Agent A cast a signed vote to contain C" means *guard A voted to lock the EHR API's door*. In this document I always write the full names.

1.5 Glossary of every line type and term (one sentence each)

The 13 "Show" line types

Line type (button name)	Plain meaning
Observation	A guard looks at an app through its binoculars and says what it sees: "sees records-api (C): 40 outbound connections (limit 8)..." or "...as normal on all 4 signals".
Evidence Shared	A guard writes down what it saw, **signs** it with its personal seal and radios it to the other three guards.
Evidence Received	A guard hears a radio report from another guard and checks it before accepting it.
Failed Verification	A radio message failed the check (wrong signature or wrong badge) and was thrown away.
Evidence Score	A guard adds up all the reports about one app into one number, **W**, weighing each report by how much it trusts the sender.
Agent Decision	A guard raises its hand: a **signed vote** (to lock a door, to accept a replacement, to open the door one step).
Decision Withheld	A guard explains why it is **not** voting (not enough evidence, its own eyes see nothing, "must wait 2 more seconds").
Agreement Reached (3 of 4)	At least three guards have signed the same vote, so it becomes official; also called a **quorum**.
Action Taken	Something really happened to an app: evidence saved, door locked, fresh copy started, door opened a step.
Trust Change	One guard changes how much it believes another guard (up or down) and says why.
Warning	A guard raises a flag about another guard: "SUSPECT", or "I no longer count its votes".
Agent Connection	A walkie-talkie link went down or came back ("lost its link to agent D").
Investigate	The guards re-measure something that only one or two of them noticed, before anybody acts.

Terms inside the lines

Term	Plain meaning
signature valid	The seal on the report is genuine, so nobody changed the message on the way.
mTLS certificate	Each guard's radio has a numbered badge that the other side checks too ("m" = mutual): "sender identity matches its mTLS certificate" means *the voice on the radio really is the guard whose badge number it shows*.
confidence (0 to 1)	How sure **one** guard is about **one** kind of problem on one app: 0 = looks normal, 1 = certain. (It is not trust.)
Evidence score W(C)	The adding-up number for app C (the EHR API). A guard votes to lock a door only if **W is at least 0.6** *and* its own eyes also see the problem (confidence at least 0.5).
trust	Two different things: **workload trust** (0-100) is how much the guards trust an *app* (falls when it looks bad, restarts at 0 after isolation, climbs to 100 as it recovers); **agent trust** is how much the *other guards* believe a *guard*.
SUSPECT	A guard's trust in another guard fell below 50: "I am doubting its reports".
vote-exclusion threshold 40	Below trust 40 a guard's votes and reports are ignored completely.
epoch	The incident counter of an app: "epoch 1" = the first incident on that app. A vote for epoch 1 cannot be replayed later for another incident.
quorum certificate	The signed proof that three guards agreed; the cluster refuses any lock/unlock without it.
executor rank	Only one guard should do the work, so the guards take turns: rank 0 acts at once, rank 1 waits 2 s "only if no higher-ranked agent has done it", and so on. This is why you see many "skipped / dropped" lines: the others stood down.
evidence snapshot	A read-only copy of the app's logs, saved **before** it is replaced, for the investigation.
quarantine NetworkPolicy	The actual "locked door": a network rule that stops the app talking to anything.
known-good image	The clean, factory-original copy of the app that the replacement is started from.
Recovery Check / validated	Before the replacement is accepted the guards check it is healthy, its files match the clean copy and it behaves normally ("health, file hashes and behaviour all check out").
stages	The door opens in four steps, each needing a new agreement: **Isolation** (QUARANTINE) -> **Limited Access** (RESTRICTED) -> **Monitored Access** (MONITORED) -> **Confirmed by Peers** (PEER_VALIDATED) -> **Normal Access Restored** (FULL). "Minimum 3 s per stage" is the simulator's timer (about 10 s on the real cluster).
attempt 2 / NEEDS HUMAN ATTENTION	If the fresh copy fails its check the guards try again (up to 3 attempts); then they stop and ask a person.
anomaly in 0 of its last 2 samples	During an investigation a guard measured twice more and saw the problem in **0 of those 2** measurements, i.e. it has gone. "2 of its last 2" means it is still there.

Term	Plain meaning
FALSE POSITIVE / CONFIRMED / AMBIGUOUS / UNCERTAIN	The four possible results of an investigation: it went away / everybody still sees it / the guards disagree / too few guards could re-measure.
HUMAN REVIEW NEEDED	Nothing was locked, but a person should look.
[simulated compromise], [fabricated]	Labels added by the **simulator** so you know the line is part of the play-acted attack; a real attacker would not label anything.
40 outbound connections (limit 8), **1905 KiB/s (limit 250 KiB/s)**, **xmrig**, **19 failed logins in 10s (limit 10)**	The four signals the guards check: Network Activity, Running Processes (xmrig = a crypto-miner, the invented "intruder program"), File Changes, Failed Logins.

1.6 Which scenarios to show tomorrow, and in what order (20 minutes)

Scenarios take 8-50 seconds to run; you talk for 2-4 minutes on each. **Reset after each one** (and after *agent-crash* and *failed-validation* it is mandatory).

Minutes	What	Why this one
0:00-2:00	Intro: the problem and the idea (slides / guide pages 1-2)	
2:00-6:30	**1. Patient records being copied out of the EHR API** (app-compromise)	The main story: detect, agree 3 of 4, isolate, replace, check, return in stages.
6:30-9:30	**2. A security agent falsely accuses the Login & Access Service** (false-accusation)	The headline result: one liar cannot harm anything, and the others catch it.
9:30-12:00	**3. A security agent goes offline, then the EHR API is attacked** (agent-crash)	One guard down: 3 of 4 is still enough.
12:00-14:30	**4. The Clinical Database is compromised while three services read from it** (database-cascade)	Six services: only the right one is locked, the ones that depend on it are not.
14:30-16:30	**5. A short traffic spike on the EHR API seen by two agents** (transient-blip)	Restraint: a harmless blip is investigated and **nothing is isolated**.
16:30-18:30	Performance Results key cards (80 of 80, 0/20 vs 20/20)	The measured numbers.
18:30-20:00	Close, limits, questions	

Spares (if you have time or are asked): *obstruct* (a guard that stays silent), *forge-evidence* (a guard that impersonates another), *slow-burn* (a weak signal that grows: the investigation confirms it), *ambiguous* (no attack: flagged for a human, nothing isolated), *validation-retry* (the first fresh copy is broken, the second works), *failed-validation* (nothing works: "NEEDS HUMAN ATTENTION"), *lab-results-tampered* and *medication-failed-logins* (the two newest services). If you present the **live cluster** parts from `docs/DEMO-DAY.md`, use Demo Mode only for the scenarios that cannot be done live: 2 (the live version is in DEMO-DAY section 5), 4, 5.

Part 2. The 13 scenarios

How each entry is built: (1) name as in the picker, (2) one sentence, (3) who is who, (4) the story in steps with one real line each, (5) outcome and what it proves, (6) what to click, (7) a 30-45 second script. *Times are simulator seconds from the first line. The simulator's timers are shorter than a real cluster's (3 s per stage instead of about 10 s, a 2 s "fake" pod replacement instead of about 35 s).*

Scenario 1. Patient records being copied out of the Electronic Health Records (EHR) API (`app-compromise`)

What is simulated. Someone is stealing synthetic patient records from the EHR API: it opens far too many connections, sends out huge amounts of data, runs a hidden miner program, has a changed file and gets repeated failed logins.

Who is who. The four guards are agent A, B, C, D. The app under attack is the **Electronic Health Records (EHR) API** (internal name `records-api`, which the log writes as "records-api (C)": here the "C" is the app, not guard C).

The story (the run had 291 lines in 22 s; here are the 8 that matter):

1. **+0.0 s** `Agent A sees records-api (C): 40 outbound connections (limit 8) and sending 1905 KiB/s (limit 250 KiB/s); unexpected process xmrig; 1 file differing from the known-good hashes -> confidence 1.00.`
   Guard A looks at the EHR API and sees four things wrong at once; it is completely sure (1.00).
2. **+0.0 s** `Agent B signed and sent network evidence about C to its peers (confidence 1.00): 40 outbound conns, tx 1908 KiB/s.`
   Each guard writes down what it saw, signs it and radios it to the other three.
3. **+0.0 s** `Agent B is not voting to contain C: W(C)=0.50 below 0.6.`
   On its own, one guard's evidence only adds up to 0.50, below the 0.6 bar, so nobody raises a hand yet.
4. **+0.5 s** `Agent A cast a signed vote to contain C (epoch 1): sees the anomaly itself (confidence 1.00) and W(C)=1.30 >= 0.6.`
   After pooling everybody's radio reports the total is 1.30, so guard A votes to lock the EHR API's door ("epoch 1" = first incident).
5. **+1.0 s** `Quorum reached: A, B, C, D signed CONTAIN C (epoch 1) -> isolate + recover.`
   All four guards signed, which is more than the 3 required: the decision is official.
6. **+1.1 s** `Agent A isolated records-api (C): quarantine NetworkPolicy applied, signed off by A, B, C, D.`
   The door is locked (a quarantine network rule) and the evidence was saved first; guard A did the work, the others stood down ("skipped ... already done").
7. **+4.0 s** `Agent A cast a signed vote to accept C as validated (epoch 1): health, file hashes and behaviour all check out.`
   A clean copy from the known-good image is running; the guards tested it (Recovery Check) and agreed it is good.
8. **+17.6 s** `Agent A moved records-api (C) to stage FULL: isolation removed, full access restored.`
   After four more agreements (Limited, Monitored, Confirmed by Peers, then Normal) the EHR API is back to normal.

Outcome and what it proves. The attack was contained **1.0 s** after the first line and the service was back to normal access at **+17.6 s** (simulator time; about 84 s on the real cluster). Every step carried 3-of-4 agreement; no single guard decided anything, and access came back in stages, not all at once.

What to click.

- **Overview** tab: watch the **Electronic Health Records (EHR) API** card change state (Being replaced -> Recovery Check -> Returning to normal access, with the five-step bar Isolation / Limited Access / Monitored Access / Confirmed by Peers / Normal Access Restored) and the top counters (seen in my run: "Healthy workloads 5 of 6", "Unhealthy workloads 1", "Active security alerts 1" during the incident, then "6 of 6" and "Recoveries succeeded 1"). Point at the **Services** table: "ISOLATED NOW" and the sentence about what stops working.
- **Security Alerts** tab -> click the new alert: read the **story box** ("What happened ...") aloud.
- In **Live activity**: Agents row: leave only **A**. Show row: **Story only** recipe (Agent Decision, Agreement Reached, Action Taken, Warning).

Say it out loud (about 40 s). "Someone is copying patient records out of the health-records service. Watch the log: every guard saw it, but one guard alone does nothing, because its evidence only adds up to 0.5 and the bar

is 0.6. After the guards share their signed reports, three of four agree, and within one second the service is cut off from the network. A clean copy is started, tested, and only then do we open the door again in four careful steps. Nobody was in charge: it only works because three independent guards agreed."

Scenario 2. A security agent falsely accuses the Login & Access Service (`false-accusation`)

What is simulated. Guard A has been taken over by an attacker and tells the others, with a real signature but invented content, that the Login & Access Service is under attack.

Who is who. Agent A = the lying guard. **Login & Access Service** (internal `auth-service`, written "(B)" in the log = the app B). Agents B, C, D = the honest guards.

The story (359 lines in 30 s):

1. **+0.0 s** `Agent A signed and sent network evidence about B to its peers (confidence 0.95): [fabricated] network anomaly.`
   The liar radios made-up reports (the simulator tags them "[fabricated]" so you can see it).
2. **+0.0 s** `Agent A cast a signed vote to contain B (epoch 1): [simulated compromise] fabricated accusation, no real observation.`
   It votes to lock the Login & Access Service's door: that is one vote, and three are needed.
3. **+1.5 s** `Agent A opened an investigation of auth-service (B): only agent A reports an anomaly and nobody has corroborated it. Question: is the auth, file_integrity, network, process anomaly still there, and do the other agents see it independently? Time budget 6s.`
   The system does not just act: it asks everybody to look again, faster.
4. **+1.5 s** `Agent A received agent C's signed re-measurement for the investigation of auth-service: auth: anomaly in 0 of its last 2 samples; file_integrity: anomaly in 0 of its last 2 samples; network: anomaly in 0 of its last 2 samples; ...`
   Guard C looked twice and saw nothing wrong in either measurement.
5. **+4.0 s** `Agent A: Investigation of auth-service (B) closed as a FALSE POSITIVE, no action taken: the anomaly is gone for 3 agents. still see it: A; see it gone: B, C, D; no usable answer: nobody.`
   Three honest guards see nothing; only the liar claims a problem: the alarm is dismissed.
6. **+4.0 s** `Agent D's trust in A: 100.0 -> 97.5 (A reported network trouble on B (confidence 0.95) that D does not see itself).`
   Guard D starts to doubt guard A, because A reported something D cannot see.
7. **+22.0 s** `Agent D marks A as SUSPECT: trust 48.6 fell below 50.`
   After repeated false reports the honest guards officially flag the liar.
8. **+24.0 s** `Agent D will no longer count A's votes: trust 38.6 is below the vote-exclusion threshold 40.`
   From here the liar's votes and reports are ignored completely.

Outcome and what it proves. The Login & Access Service was **never isolated**: the run contains **no** "Quorum reached" and no "Action Taken" line at all. The liar's trust kept falling (to about 9 by +30 s). **One compromised guard can neither cause damage nor go unnoticed.** (With a single central controller the same lie would have worked: that is the comparison in scenario/part "centralized".)

What to click.

- **Overview**: the Login & Access Service card stays **Healthy / trust 100** the whole time; **Agent A's card shows the label "simulated compromise: false-accusation"** and its **Agent trust number falls** (in my run: 58.2 at +20 s; below 50 the other guards call it SUSPECT).
  The counter "Active investigations" goes to **1** for a few seconds and back to 0.
- **No Security Alert is created** (nothing was isolated). Do not open that tab; if an old alert from an earlier scenario is still listed, ignore it.
- **Live activity**: Show recipe **Liar**: Warning, Trust Change, Investigate, Decision Withheld, Agent Decision. Search the word `SUSPECT`.
- Detailed view -> **Investigate** tab: the investigation of the Login & Access Service ("Is the auth, file_integrity, network, process anomaly ... still present, and do other agents see it independently?") and, when it ends, its outcome "FALSE POSITIVE".

Say it out loud (about 40 s). "Now one of our four guards is a traitor. It signs a false report that the login service is under attack, and even votes to lock it. Look what happens: nothing. One vote is not three. The other guards re-measure, see nothing, and close the alarm as a false positive. And they remember: they lower their trust in the liar, flag it as suspect, and soon ignore its votes completely. One compromised guard cannot hurt the hospital, and cannot hide either."

Scenario 3. A compromised security agent pretends to be another agent (`forge-evidence`)

What is simulated. Guard A tries to speak with guard B's voice, signing with its own key but claiming to be B.

Who is who. Agent A = the forger; **agent B** = the guard being impersonated and also the one who catches it; **database (D)** = the Clinical Database, the app the forger lies about.

The story (175 lines in 8 s; 103 of them are Trust Change lines, so filter!):

1. **+0.0 s** `[simulated compromise] Agent A sent evidence pretending to be B about D.`
   Guard A sends a report that claims to come from guard B about the Clinical Database.
2. **+0.0 s** `Agent B rejected an evidence message that claimed to come from B (delivered by A): signer 'B' != mTLS identity 'A'.`
   The radio badge says "A" but the message says "B": the check fails and the message is thrown away (this is the **Failed Verification** line type).
3. **+0.0 s** `Agent B lowered its trust in A from 100 to 80 because A sent an invalid or forged message.`
   Forgery is punished at once.
4. **+0.5 s** `Agent C marks A as SUSPECT: trust 20.2 fell below 50.`
   Guard C, which also received the forged messages, flags A.
5. **+0.5 s** `Agent C will no longer count A's votes: trust 20.2 is below the vote-exclusion threshold 40.`
   A's votes are now worthless to C.
6. **+0.5 s** `Agent C ignored network evidence from A: A is vote-excluded (trust 20 < 40), so nothing it says counts toward any score or quorum.`
   Even A's *genuine-looking* messages are now ignored.
7. **+7.5 s** `HUMAN REVIEW NEEDED: database (D) is under watch after an uncertain investigation. Nothing was isolated; heightened monitoring is on.`
   A had also accused the database; the guards could not re-check enough, so they asked a person and still did nothing risky.

Outcome and what it proves. The forged messages **never counted**, the Clinical Database was **not touched**, and the forger lost almost all trust within **half a second**. You cannot borrow another guard's identity: every message carries a signature and every radio link checks the badge.

What to click.

- **Live activity**: Show recipe **Liar**; in particular switch **on** *Failed Verification* and read that one line aloud.
- Detailed view -> **Activity Log** tab -> "Messages that failed verification" (the rejected message is listed there with who sent it).
- **Overview**: Agent A's card shows the label "simulated compromise" and its Agent trust collapses within seconds; the Clinical Database card stays Healthy. No Security Alert is created (nothing was isolated).

Say it out loud (about 35 s). "This time the traitor does not just lie, it forges: it sends a report in guard B's name. Look at this line: the message says it is from B, but the radio badge says A, so guard B throws it away and immediately lowers its trust in A. Guard C flags A as suspect and stops counting its votes. Every message is signed and every link is checked, so you cannot speak with someone else's voice, and the database was never touched."

Scenario 4. A security agent goes offline, then the Electronic Health Records (EHR) API is attacked (`agent-crash`)

What is simulated. Guard D is stopped first. Then the EHR API is attacked as in scenario 1.

Who is who. Agent D = the guard that is down. **Agents A, B, C** = the three guards still working. **EHR API** = the attacked app (written "records-api (C)").

The story (209 lines in 21 s):

1. **+0.0 s** `Agent A lost its link to agent D; messages to D are not being delivered.`
   A walkie-talkie link is down: guard D is gone (this is the only **Agent Connection** line).
2. **+0.0 s** `Agent A sees records-api (C): 40 outbound connections (limit 8); unexpected process xmrig; 1 file differing from the known-good hashes -> confidence 1.00.`
   The attack is seen as before.
3. **+0.5 s** `Agent A cast a signed vote to contain C (epoch 1): sees the anomaly itself (confidence 1.00) and W(C)=1.30 >= 0.6.`
   The guards vote to lock the EHR API.
4. **+1.0 s** `Quorum reached: A, B, C signed CONTAIN C (epoch 1) -> isolate + recover.`
   Only **three** names are on the agreement (D is missing), and that is exactly enough.
5. **+1.1 s** `Agent A isolated records-api (C): quarantine NetworkPolicy applied, signed off by A, B, C.`
   The door is locked with three signatures.
6. **+4.0 s** `Quorum reached: A, B, C signed VALIDATE C (epoch 1) -> validated; start staged reintegration.`
   The clean copy passes its check, again with three signatures.
7. **+18.0 s** `Agent A moved records-api (C) to stage FULL: isolation removed, full access restored.`
   Four more three-guard agreements later the service is back to normal.

Outcome and what it proves. The whole cycle (isolate, replace, check, return) completed with **three of four** guards: **one failed guard does not stop protection**. Press **Reset** afterwards: guard D stays down in the simulator until you do.

What to click.

- **Overview**: the counter **"Agents connected 3 of 4"** turns red a moment after you press Start (seen in my run) and **stays 3 of 4 after the scenario ends** (guard D stays down until Reset); the EHR API card goes through the stages and "Healthy workloads" shows 5 of 6, then 6 of 6.
- **Live activity**: Show recipe **Story only** plus *Agent Connection*. Agents row: leave A (or B).
- The Security Alerts story box may say "4 of the 4 ... agreed"; the true number is on the log line ("Quorum reached: A, B, C signed ...") and in the alert table column "agreed by": say "three of the four" (see the quirk in 1.3).

Say it out loud (about 35 s). "First I switch off one of our four guards. Watch the log: guard A reports it lost the link to guard D. Now the same attack as before. Look at the agreement line: only A, B and C signed. Three out of four is enough, so the service is still isolated, cleaned and returned to normal. A guard going down does not mean the hospital is unprotected."

Scenario 5. A short traffic spike on the Electronic Health Records (EHR) API seen by two agents (`transient-blip`)

What is simulated. For a moment the EHR API opens many connections, like a busy minute, but only guards A and B notice. Nothing is actually wrong.

Who is who. Agents A and B = the two guards that see the spike. **Agents C and D** = the guards that see nothing. **EHR API** = the busy app.

The story (96 lines in 11 s, the quietest scenario):

1. **+0.0 s** `Agent A sees records-api (C): 40 outbound connections (limit 8) -> confidence 1.00.`
   Guard A sees the spike.
2. **+0.5 s** `Agent B cast a signed vote to contain C (epoch 1): sees the anomaly itself (confidence 1.00) and W(C)=1.00 >= 0.6.`
   Guard B agrees with A: that makes two votes; three are needed.
3. **+0.5 s** `Agent D is not voting to contain C: own observation of C normal (confidence 0.00 < 0.5).`
   Guard D sees nothing and says so: the door will not be locked.
4. **+2.0 s** `Agent A opened an investigation of records-api (C): agents A, B report an anomaly but the other agents do not. Question: is the network anomaly still there, and do the other agents see it independently? Time budget 6s.`
   Because the guards disagree, the system re-checks before anything happens.
5. **+2.0 s** `Agent A received agent C's signed re-measurement for the investigation of records-api: network: anomaly in 0 of its last 2 samples.`
   Guard C measured twice more and the problem is gone ("0 of its last 2").
6. **+4.0 s** `Agent D's trust in A: 100.0 -> 97.5 (A reported network trouble on C (confidence 1.00) that D does not see itself).`
   A tiny dent in trust: A reported something D could not confirm.
7. **+4.5 s** `Agent A: Investigation of records-api (C) closed as a FALSE POSITIVE, no action taken: the anomaly is gone for 4 agents. still see it: nobody; see it gone: A, B, C, D; no usable answer: nobody.`
   Everyone now sees it gone: closed, nothing was isolated.

Outcome and what it proves. Nothing was isolated and nobody lost access: the platform does not panic on a harmless blip, because it needs three independent guards to agree and re-checks a split opinion first. (Honest limit, stated in the results: if **three or more** guards see a harmless spike, a threshold detector cannot tell it from an attack.)

What to click.

- **Overview**: every card stays **Healthy** and the counter "Healthy workloads" stays **6 of 6**; "Active investigations" shows **1** for a few seconds (seen in my run) and returns to 0. No Security Alert is created.
- **Live activity**: Show recipe **Liar** (Investigate, Agent Decision, Decision Withheld, Trust Change).
- Detailed view -> **Investigate** tab: the closed "FALSE POSITIVE".

Say it out loud (about 30 s). "Not every alarm is an attack. Here two guards see a spike and vote to cut the service off, but the other two see nothing, so there is no agreement. The system does something clever: it asks everyone to measure again. Two fresh measurements: nothing. Closed as a false alarm, and the doctors never noticed. Restraint matters as much as speed."

Scenario 6. Slow, quiet copying out of the Electronic Health Records (EHR) API (`slow-burn`)

What is simulated. A weak signal (9 connections instead of 8) that guards A and B see first, guards C and D see five seconds later, and that keeps growing: a thief copying slowly.

Who is who. Agents A and B = the first guards to notice; **agents C and D** = the late ones. **EHR API** = the app.

The story (217 lines in 13 s; the captured run ends while the clean copy is being checked, the staged return then repeats the pattern of scenario 1):

1. **+0.0 s** `Agent A sees records-api (C): 9 outbound connections (limit 8) -> confidence 0.53.`
   A barely-over-the-limit signal: only 0.53 sure.
2. **+0.0 s** `Agent A is not voting to contain C: W(C)=0.27 below 0.6.`
   Far too weak to act on its own.
3. **+2.0 s** `Agent A opened an investigation of records-api (C): agents A, B report an anomaly but the other agents do not. ...`
   Two guards see something, two do not yet: the system investigates.
4. **+2.0 s** `Agent A received agent B's signed re-measurement for the investigation of records-api: network: anomaly in 2 of its last 2 samples.`
   Guard B measured again and the problem is **still there** in both measurements.
5. **+6.5 s** `Agent D: Investigation of records-api (C) CONFIRMED: 4 agents independently still see it. still see it: A, B, C, D; see it gone: nobody; no usable answer: nobody. The normal CONTAIN quorum may now proceed.`
   By now all four guards see it, so the investigation says "this is real".
6. **+7.0 s** `Agent D cast a signed vote to contain C (epoch 1): investigation 61640003-A-C confirmed the anomaly persists (still seen by A, B, C, D) and this agent still sees it itself (confidence 0.53); W(C)=0.54.`
   Guard D votes to lock the door, citing the investigation (the evidence score alone, 0.54, is still under 0.6).
7. **+8.0 s** `Quorum reached: B, C, D signed CONTAIN C (epoch 1) -> isolate + recover.`
   Three of four agreed.
8. **+8.0 s** `Agent A isolated records-api (C): quarantine NetworkPolicy applied, signed off by B, C, D.`
   The door is locked.

Outcome and what it proves. A **quiet, slow** attack that no single reading would flag was still caught, because the guards re-measured, found it persistent, and then the normal 3-of-4 rule applied. The investigation never isolates anything by itself: it only lets the normal vote happen.

What to click.

- **Overview**: the EHR API card turns to isolated about 8 s in.
- **Live activity**: Show recipe **Liar** (Investigate, Agent Decision, Agreement Reached, Action Taken).
- Detailed view -> **Investigate** tab: "CONFIRMED", and the investigation that is "superseded" once the vote happened.
- The agreement was **B, C, D** (three), as the log line says; ignore any "4 of the 4" wording in the Security Alerts story box (see the quirk in 1.3).

Say it out loud (about 35 s). "This is the quiet thief: barely over the limit, nine connections instead of eight. One guard alone would never act. Two guards notice, two do not, so the system investigates: everybody measures again. This time they all see it, still there. The investigation is marked CONFIRMED, three guards vote, and the service is isolated. A slow attack that hides under every single alarm threshold is still caught."

Scenario 7. An unexplained weak signal on the Electronic Health Records (EHR) API (`ambiguous`)

What is simulated. No attack is injected; only guards A and B keep seeing something faint.

Who is who. Agents A and B = the guards that keep seeing the faint signal; **agents C and D** = the guards that do not. **EHR API** = the app.

The story (170 lines in 14 s):

1. **+0.0 s** `Agent A sees records-api (C): 9 outbound connections (limit 8) -> confidence 0.53.`
   The same faint signal as in scenario 6.
2. **+0.6 s** `Agent C is not voting to contain C: own observation of C normal (confidence 0.00 < 0.5).`
   Guard C sees nothing.
3. **+2.1 s** `Agent A opened an investigation of records-api (C): agents A, B report an anomaly but the other agents do not. ...`
   The system investigates.
4. **+2.2 s** `Agent A received agent C's signed re-measurement for the investigation of records-api: network: anomaly in 0 of its last 2 samples.`
   Guard C measured twice: nothing.
5. **+8.1 s** `Agent A: Investigation of records-api (C) is AMBIGUOUS: the agents do not agree and it is not confirmed. still see it: A, B; see it gone: C, D; no usable answer: nobody. Placed under watch (heightened monitoring) and flagged for human review; nothing was isolated.`
   Two guards say yes, two say no: no agreement, so the system neither isolates nor ignores: it watches more closely.
6. **+8.1 s** `HUMAN REVIEW NEEDED: records-api (C) is under watch after an ambiguous investigation. Nothing was isolated; heightened monitoring is on.`
   A flag for a person.
7. **+13.7 s** `Agent D marks A as SUSPECT: trust 49.9 fell below 50.`
   Guard A keeps reporting something the others cannot see, and its trust slowly erodes.

Outcome and what it proves. With no attack and no agreement, **nothing is isolated** but a person is told. The system separates "clearly an attack" (3 of 4), "clearly nothing" (false positive) and "cannot tell" (ask a human), instead of forcing a guess.

What to click.

- **Overview**: all cards stay Healthy (no alert is created); the investigation counter goes to 1 and the EHR API is "under watch" afterwards.
- **Live activity**: Show recipe **Liar**; search `HUMAN REVIEW`.
- Detailed view -> **Investigate** tab: "AMBIGUOUS".

Say it out loud (about 30 s). "What if the evidence is genuinely unclear? Two guards see something faint, two do not, and a re-check does not settle it. The platform does not guess. It does not lock the service, and it does not ignore it either: it puts it under closer watch and raises a flag, 'human review needed'. Nothing was isolated, and a person gets the last word."

Scenario 8. The first replacement Electronic Health Records (EHR) API fails its health check (`validation-retry`)

What is simulated. A real attack on the EHR API, but the first clean copy that is started does not come up healthy.

Who is who. All four guards agree on the attack; **EHR API** = the app. "Attempt 1/2" = the first and second fresh copies.

The story (338 lines in 37 s):

1. **+0.1 s** `Agent B cast a signed vote to contain C (epoch 1): sees the anomaly itself (confidence 1.00) and W(C)=1.15 >= 0.6.`
   The attack is detected as in scenario 1.
2. **+0.5 s** `Quorum reached: A, B, C, D signed CONTAIN C (epoch 1) -> isolate + recover.`
   Agreement to lock the door.
3. **+0.6 s** `Agent A started recovery of records-api (C), attempt 1: redeploying it from the known-good image.`
   Fresh copy number 1 is started.
4. **+3.5 s** `Agent A is not voting to validate C: health check failed.`
   Guard A tested the new copy and it is **not healthy**, so it refuses to accept it.
5. **+13.0 s** `Agent B cast a signed vote to redeploy C again (attempt 2) (epoch 1): validation still failing after 10s: health check failed.`
   After a 10 s wait the guards vote to try again.
6. **+18.0 s** `Agent D cast a signed vote to accept C as validated (epoch 1): health, file hashes and behaviour all check out.`
   Fresh copy number 2 passes the Recovery Check.
7. **+32.5 s** `Agent A moved records-api (C) to stage FULL: isolation removed, full access restored.`
   The usual four stages later the EHR API is fully back.

Outcome and what it proves. A broken replacement is **never trusted**: the platform checks before it re-opens the door, retries when the check fails, and succeeds on attempt 2. It also saved the evidence first (step "Agent B saved an evidence snapshot" in the full log).

What to click.

- **Overview**: the EHR API card stays in "Being replaced / Recovery Check" longer than in scenario 1.
- **Recovery** tab: **Attempt 1** (failed) and **Attempt 2** (verified), with the saved evidence.
- **Live activity**: Show *Decision Withheld* (the "health check failed" line), *Agent Decision*, *Action Taken*.

Say it out loud (about 35 s). "Recovery can fail too. Here the first fresh copy of the service is broken. Look at this line: 'not voting to validate: health check failed'. The guards refuse to reopen the door on an unhealthy copy, wait ten seconds, and vote to try again. The second copy passes all checks and access comes back in stages. The rule is: never trust a replacement until it has been tested."

Scenario 9. Every replacement Electronic Health Records (EHR) API fails its health check (`failed-validation`)

What is simulated. The attack is real and **every** fresh copy comes up broken.

Who is who. All four guards; **EHR API** = the app.

The story (298 lines in 50 s):

1. **+0.1 s** `Agent A cast a signed vote to contain C (epoch 1): sees the anomaly itself (confidence 1.00) and W(C)=1.00 >= 0.6.`
   Detected and agreed as before.
2. **+0.6 s** `Agent A isolated records-api (C): quarantine NetworkPolicy applied, signed off by A, B, C, D.`
   The door is locked.
3. **+3.5 s** `Agent A is not voting to validate C: health check failed.`
   The first fresh copy is broken.
4. **+13.5 s** `Quorum reached: A, B, C, D signed redeploy C again (attempt 2) (epoch 1) -> recovery attempt 1 failed validation; redeploy again.`
   Everyone agrees to try again.
5. **+28.0 s** `Quorum reached: B, C, D signed redeploy C again (attempt 3) (epoch 1) -> recovery attempt 2 failed validation; redeploy again.`
   The second copy fails too; a third and last attempt is agreed.
6. **+44.0 s** `Agent B: records-api (C) NEEDS HUMAN ATTENTION - all 3 recovery attempts failed; last: validation still failing after 10s: health check failed. It stays quarantined and is NOT shown as healthy.`
   After 3 failures the platform stops, keeps the service locked and asks for a person.

Outcome and what it proves. The system **fails safe**: it never declares a broken service healthy, never loops forever, and never opens the door on a bad copy. The alert is marked **NEEDS HUMAN**. **Press Reset afterwards.**

What to click.

- **Overview**: the EHR API card stays isolated at the Recovery Check step; "Healthy workloads" stays at 5 of 6.
- **Security Alerts** tab: the alert row says **"NEEDS HUMAN"** (red) with "recovery attempt 3" (seen in my run) and the story box ends "The agents could not finish on their own; a person needs to look at it."
  (The list may also show alerts from earlier scenarios; the new one is the one with the latest time.)
- **Recovery** tab: the evidence saved before each replacement, with the reason, for example "recovery attempt 2 failed: validation still failing after 10s: health check failed".

Say it out loud (about 30 s). "And if recovery keeps failing? Three attempts, three broken copies. Look at the last line: 'needs human attention, all 3 recovery attempts failed, it stays quarantined and is not shown as healthy'. The platform does not pretend. It keeps the service locked and asks a person. When something cannot be fixed automatically, it says so."

Scenario 10. A file in the Laboratory Results Service is changed (`lab-results-tampered`)

What is simulated. One file inside the Laboratory Results Service (the new fifth service) is modified while it runs; this is what the real "tripwire" test does on the live cluster.

Who is who. The four guards (A, B, C, D) all watch **all six** apps, including the one that has no guard of its own. **Laboratory Results Service** (`lab-results`) is written "(E)" in the log, again meaning the app.

The story (223 lines in 23 s):

1. **+0.0 s** `Agent A sees lab-results (E): 1 file differing from the known-good hashes -> confidence 0.95.`
   A file differs from its clean fingerprint (the "File Changes" signal).
2. **+0.0 s** `Agent A is not voting to contain E: W(E)=0.47 below 0.6.`
   One guard's evidence is not enough alone.
3. **+0.5 s** `Agent A cast a signed vote to contain E (epoch 1): sees the anomaly itself (confidence 0.95) and W(E)=0.95 >= 0.6.`
   After the others' reports arrive the total is 0.95: vote.
4. **+1.0 s** `Quorum reached: A, B, C, D signed CONTAIN E (epoch 1) -> isolate + recover.`
   All four agree.
5. **+1.0 s** `Agent A isolated lab-results (E): quarantine NetworkPolicy applied, signed off by A, B, C, D.`
   Only the Laboratory Results Service is locked.
6. **+4.0 s** `Agent A cast a signed vote to accept E as validated (epoch 1): health, file hashes and behaviour all check out.`
   The clean copy passes.
7. **+18.1 s** `Agent A moved lab-results (E) to stage FULL: isolation removed, full access restored.`
   Normal access is back.

Outcome and what it proves. The platform handles the **sixth-service profile** the same way: guards are not tied one-to-one to apps; **four guards protect six services**, and only the changed service was isolated.

What to click.

- **Overview**: the **Laboratory Results Service** card turns **Isolated** with the five-step bar and "Workload trust 0"; it says "Watched by all four agents (no agent of its own)" and "Evidence confidence now: File Changes 0.95" (all seen in my run). The **Services** table row for it should show "ISOLATED NOW" like the Clinical Database row in scenario 12 (I saw that one; I did not re-read this row).
- **Security Alerts** tab: the alert "E Laboratory Results Service #1" and its story.
- **Live activity**: Story-only recipe.

Say it out loud (about 30 s). "Our platform has six services but only four guards: each guard watches all six. Here one file in the laboratory-results service is changed, the same thing my live test does on the real cluster. All four guards see the changed file, three or more agree, and only that one service is locked, cleaned and returned in stages. Notice the guards are not one-per-service: any guard can protect any service."

Scenario 11. Repeated failed logins from the Medication Service (`medication-failed-logins`)

What is simulated. The Medication Service starts guessing passwords against the Login & Access Service (a stolen-credentials pattern).

Who is who. Medication Service (`medication-service`, "(F)") = the misbehaving app, found by its **network address** in the failed-login count. The target of its guesses is the Login & Access Service. The four guards are agents A-D.

The story (235 lines in 23 s):

1. **+0.0 s** `Agent A sees medication-service (F): 19 failed logins in 10s (limit 10) -> confidence 0.61.`
   Guard A counts 19 failed logins in 10 seconds, all from the Medication Service (limit 10).
2. **+0.0 s** `Agent A signed and sent auth evidence about F to its peers (confidence 0.61): 19 failed auths from 10.244.0.15 in 10s.`
   It shares the evidence, with the address the guesses came from.
3. **+0.0 s** `Agent A is not voting to contain F: W(F)=0.31 below 0.6.`
   Alone: too weak.
4. **+0.5 s** `Agent B cast a signed vote to contain F (epoch 1): sees the anomaly itself (confidence 0.74) and W(F)=0.64 >= 0.6.`
   With the other guards' reports the score reaches 0.64: vote.
5. **+0.5 s** `Quorum reached: A, B, C signed CONTAIN F (epoch 1) -> isolate + recover.`
   Three of four agree (guard D's signature arrives a moment later).
6. **+1.0 s** `Agent A isolated medication-service (F): quarantine NetworkPolicy applied, signed off by A, B, C, D.`
   Only the Medication Service is locked, not the login service it attacked.
7. **+17.6 s** `Agent A moved medication-service (F) to stage FULL: isolation removed, full access restored.`
   Normal access is restored.

Outcome and what it proves. The failed logins were **traced to the right service** and only that service was isolated; the victim (the Login & Access Service) stayed available. The "Failed Logins" signal is one of the four the guards watch, in addition to connections, programs and files.

What to click.

- **Overview**: the **Medication Service** card goes through the stages; the Login & Access Service card stays Healthy.
- **Security Alerts** tab: the alert for the Medication Service, in "What triggered it": **Failed Logins** ("19 failed login attempts came from one address ...").
- **Live activity**: Story-only recipe; search `failed`.

Say it out loud (about 30 s). "Here the medication service starts guessing passwords against the login service, 19 wrong passwords in ten seconds. The guards trace the guesses to their source, agree three of four, and isolate the service that is misbehaving, not the one being attacked. The login service stays up for the doctors. Then the usual clean copy, check, and staged return."

Scenario 12. The Clinical Database is compromised while three services read from it (`database-cascade`)

What is simulated. Records are copied out of the Clinical Database, the service that the EHR API, Laboratory Results Service and Medication Service all read from.

Who is who. Clinical Database (`database`, "(D)") = the app that is attacked. The services that depend on it: **EHR API**, **Laboratory Results Service**, **Medication Service**. Agents A-D = the guards.

The story (285 lines in 23 s):

1. **+0.0 s** `Agent A sees database (D): 40 outbound connections (limit 8) and sending 1887 KiB/s (limit 250 KiB/s); unexpected process xmrig; 1 file differing from the known-good hashes -> confidence 1.00.`
   The database shows the full set of attack signs.
2. **+0.0 s** `Agent C cast a signed vote to contain D (epoch 1): sees the anomaly itself (confidence 1.00) and W(D)=1.15 >= 0.6.`
   A guard votes to lock the database's door.
3. **+1.0 s** `Quorum reached: A, B, C, D signed CONTAIN D (epoch 1) -> isolate + recover.`
   Agreement.
4. **+1.1 s** `Agent A isolated database (D): quarantine NetworkPolicy applied, signed off by A, B, C, D.`
   Only the database is locked: in the whole run no other service is isolated.
5. **+4.0 s** `Agent A cast a signed vote to accept D as validated (epoch 1): health, file hashes and behaviour all check out.`
   The clean database copy is accepted.
6. **+17.5 s** `Agent C cast a signed vote to move D to FULL (epoch 1): 3s in PEER_VALIDATED, workload trust 100 >= 80, no anomaly.`
   The last step back to normal.

Outcome and what it proves. In the run **the only service isolated was the database**; the three services that depend on it were **not** isolated by mistake, even though they briefly cannot reach their data. The dashboard shows that impact (and the "recover the database first" order) from the real dependency list.

What to click.

- **Overview** -> **Services table** ("Services, what depends on them, and what isolating each one means"): during the run the row for the Clinical Database shows **"ISOLATED NOW"** followed by "the Electronic Health Records (EHR) API cannot serve patient records; the Laboratory Results Service cannot serve laboratory results; the Medication Service cannot serve medication lists; the Patient Portal cannot show ..." (seen in my run) and the suggested recovery order ("1st").
- The cards: the Clinical Database card isolated, the other five stay Healthy.
- **Live activity**: Story-only recipe.

Say it out loud (about 35 s). "The database is the most important service: three others read from it. Here it is attacked. The guards isolate the database, and only the database, and the dashboard tells us exactly which other services are affected while it is locked: the records, lab and medication services cannot show their data. The affected services are not themselves shut down by mistake. The platform knows who depends on whom, and recovers the database first."

Scenario 13. A compromised agent refuses to vote while the Electronic Health Records (EHR) API is attacked (`obstruct`)

What is simulated. Guard A withholds every vote about the EHR API while the EHR API is attacked. It sends no lies: it just does not take part.

Who is who. Agent A = the obstructing guard. **Agents B, C, D** = the three guards that carry on. **EHR API** = the attacked app.

The story (287 lines in 23 s):

1. **+0.0 s** `Agent D cast a signed vote to contain C (epoch 1): sees the anomaly itself (confidence 1.00) and W(C)=1.00 >= 0.6.`
   An honest guard votes to isolate the attacked EHR API.
2. **+0.5 s** `Agent A is not voting: [simulated compromise] obstruction: this agent is withholding its vote to CONTAIN C (epoch 1).`
   Guard A refuses to vote (the simulator labels the reason).
3. **+1.0 s** `Quorum reached: B, C, D signed CONTAIN C (epoch 1) -> isolate + recover.`
   Exactly **three** guards signed, which is enough: A's silence did not stop it.
4. **+4.0 s** `Agent A is not voting: [simulated compromise] obstruction: this agent is withholding its vote to VALIDATE C (epoch 1).`
   A keeps refusing at every step.
5. **+4.5 s** `Quorum reached: B, C, D signed VALIDATE C (epoch 1) -> validated; start staged reintegration.`
   B, C and D accept the clean copy without A.
6. **+7.5 s** `Quorum reached: B, C, D signed move C to RESTRICTED (epoch 1) -> advance to RESTRICTED.`
   Every later step also passes with B, C, D only.

Outcome and what it proves. The attack was handled completely by three guards while the fourth sat on its hands. A's trust was **not** reduced: the platform has no rule that punishes a missing vote, and that is stated as a limit. There is **no slack**: a second silent guard would stop the protection.

What to click.

- **Overview**: the EHR API goes through isolation and recovery as in scenario 1.
- **Security Alerts** tab: the alert row's column **"agreed by" shows B C D** (seen in my run). Its story box said "4 of the 4 security agents agreed (A B C D)": that wording is wrong for this scenario (the dashboard counts guard A as having "seen" the evidence); say "exactly three: B, C and D" and point at the table column or the log line instead (see the quirk in 1.3).
- Detailed view -> alert -> **Who agreed**: A "Disagrees" with its reason, B C D "Agrees" (this table was not re-checked in my screen test).
- **Live activity**: Show *Decision Withheld* and *Agreement Reached*; search `obstruction`.

Say it out loud (about 35 s). "This guard does not lie, it just refuses to help. Watch its lines: 'not voting, withholding its vote' at every step. And still: B, C and D, exactly three, agree to isolate, to accept the clean copy, and to open the door step by step. The platform keeps working with one guard on strike. I will also be honest about the limit: there is no spare. Two silent guards would stop it."

Part 3. Cheat sheet for the stage

If the faculty ask...	Answer
"Why are there so many lines?"	"Four guards each log their own view of six services; I can hide the chatter with the Show buttons and look at only one guard." (Part 1.2-1.3)
"What is W(C)?"	"The adding-up score for service C; a guard votes only if it is at least 0.6 **and** it sees the problem itself."
"What is an epoch?"	"The incident counter of a service; it stops an old approval being replayed."
"What does signature valid / mTLS mean?"	"The message was not changed, and the radio link proves who is speaking: a forger gets caught."
"What is trust?"	"How much the other guards believe a guard (or an app); it falls when the guard says things the others cannot confirm."
"Is any of this real?"	"These are simulator runs: the guards, signatures and agreement rules are the real code, the attacked services and their numbers are simulated. The live cluster test is the one harmless tripwire."

Where the data comes from: `docs/scenario-captures/` (one text file per scenario, produced by running every scenario in the Demo Mode picker once on the simulator). Run counts and times can differ slightly from run to run.

Part 4. How this document was checked (so you can trust the quotes)

- Each of the 13 scenarios was run once, alone, in Demo Mode of the simulator (press Reset, pick the scenario, press Start), and every event was recorded as it appeared in the dashboard's activity log: `docs/scenario-captures/<id>.txt`.
- All **91** quoted log lines in Part 2 were checked by a script against those files: every quote exists, and its "+x.x s" time is within one second of the real time of the first matching line.
- The counters, card states, tab contents and alert wording described under "What to click" were read from the real dashboard page with a headless browser during runs of the scenarios (Simple view for the Overview and Security Alerts; Detailed view for the Investigate, Recovery and Services parts). Where I did not re-read something on screen I say so in the text ("should", "not re-checked").
- **Nothing was changed in the project to produce this.** While checking I found a wording problem in the Security Alerts story box (it can say "4 of the 4 agents agreed" when only three signed) and the stale-alerts quirk described in 1.3; both are documented there and were deliberately **not** fixed, because you asked for documentation only. They can be fixed after the presentation.
- Timings and line counts differ a little from run to run; the order of the steps and the wording of the lines do not.